PjBL: Duas empresas, duas restrições

Deduplicação de leituras de sensores — relatório técnico.

Integrantes: Henry Rodrigues, Rafael Maluf e Matheus.

Fundamentação matemática

Este capítulo organiza a defesa do trabalho: primeiro o que precisa ser provado, depois quanto custa. Resultados empíricos complementam a análise; eles não substituem uma prova, e uma função assintótica não determina segundos.

1. Especificação como propriedade matemática

Para o vetor original $A=(a_0,\ldots,a_{n-1})$, defina $S(A)=\{a_i : 0\le i<n\}$. A saída é um prefixo B de comprimento u. Exigimos simultaneamente: (i) todo elemento de B pertence a S(A); (ii) cada elemento de S(A) aparece em B; (iii) os elementos de B são dois a dois distintos. Essas três propriedades implicam $u=|S(A)|$. Acertar apenas u não prova corretude: uma saída de tamanho correto pode trocar um valor por outro.

Para n=0, S é vazio e u=0. Para n>0, $1\le u\le \min(n,D)$, D=65536. O princípio das gavetas prova que n>D implica ao menos uma repetição; não prova que todos os D valores ocorreram. No modelo uniforme independente, $E[u]=D[1-(1-1/D)^n]$, mas esse valor é uma esperança, não o u de uma entrada concreta. Prova: o indicador I_x de presença de x tem esperança $1-(1-1/D)^n$; somar sobre os D valores e usar linearidade da esperança. Independência entre os indicadores não é necessária.

A leitura de n elementos é necessária no pior caso para qualquer algoritmo exato, sem certificação prévia da entrada: se uma posição não for inspecionada, pode-se alterá-la para um valor ausente e mudar a resposta. Portanto há limite inferior $\Omega(n)$ no modelo geral de acesso às leituras. Isso situa o bitmap em $\Theta(n)$ com domínio fixo, assintoticamente ótimo nesse modelo; não afirma que sua constante seja a melhor em todo hardware.

2. Prova de corretude do Heap Sort e da compactação

Invariante do sift. As subárvores dos filhos já são max-heaps; uma eventual violação está na raiz corrente. Se a raiz é maior ou igual ao maior filho, a propriedade foi restaurada. Caso contrário, trocar com o maior filho faz a nova raiz dominar os dois filhos e desloca a única violação possível para a subárvore escolhida. A altura restante diminui, logo o laço termina. Escolher o maior filho é essencial.

Construção. Folhas são heaps triviais. Ao processar nós internos em ordem decrescente de índice, as subárvores dos filhos já são heaps. Pelo argumento anterior, cada passo torna um heap a subárvore atual. Ao fim, o vetor todo é um max-heap.

Extração. Antes de cada extração, o prefixo é um max-heap e o sufixo está ordenado, com elementos maiores ou iguais aos do prefixo. A raiz é o máximo restante. Movê-la ao início do sufixo e restaurar o heap mantém o invariante. O prefixo diminui em um até terminar. Trocas conservam o multiconjunto de valores; o vetor resultante é uma permutação ordenada da entrada.

Compactação. Antes de ler o índice i, o prefixo a[0..u) contém exatamente um representante de cada valor do prefixo ordenado já lido. Se a[i] coincide com o último emitido, não surge um valor novo. Se difere, a ordenação garante que ele não apareceu em nenhum bloco anterior; anexá-lo mantém as três propriedades da saída. Como $u\le i$ na entrada de cada passo, escrever em a[u] não altera a região futura. O caso n<2 é tratado antes de acessar qualquer posição.

Tempo. Uma descida custa O(h) para altura h. Na construção, o número de nós de altura pelo menos h é no máximo $\lfloor n/2^h \rfloor$; por isso a soma dos trabalhos é limitada por $n\,\Sigma(h\ge 1)1/2^h=O(n)$, além de $\Omega(n)$ visitas. As extrações somam no máximo $\Sigma(m=2..n)O(\log m)=O(n \log n)$. Com todos iguais, cada sift interrompe após uma comparação local e o total é $\Theta(n)$. Para ordens adversas com muitas chaves distintas, $\Theta(n \log n)$ é

atingido; com D fixo esse limite superior permanece válido, mas não se usa o limite inferior de ordenação de n chaves distintas quando n>D. O custo completo soma Θ(n) de compactação.

Espaço. Há apenas o frame da função externa e um frame de sift simultâneo, sem cadeia recursiva. O número de variáveis não cresce com n; o tamanho depende da ABI. A prova estrutural é O(1); os 96 B são evidência do build local, não consequência numérica da notação O(1).

3. Prova de corretude e custo do bitmap

Antes da iteração i: (a) o bit de x está marcado se e somente se x ocorreu no prefixo original A[0..i); (b) a[0..u) contém exatamente esses valores, nas primeiras ocorrências; (c) a região ainda não lida não foi modificada. A inicialização estabelece o invariante para i=0. Se o bit está marcado, não há valor novo e não se emite. Se está desmarcado, emitir x e marcar o bit acrescenta exatamente o novo valor. A escrita ocorre no máximo na posição i; o valor corrente já foi carregado. Assim o invariante avança e, em i=n, prova as propriedades da saída e sua ordem.

O índice x>>6 varia de 0 a 1023; o deslocamento x&63 varia de 0 a 63. UINT64_C(1) torna válido o deslocamento de 63 bits. Há 1024 palavras de 8 B: 8192 B. Em modelo de palavra w=64, $T(n,D)=\Theta(\text{ceil}(D/w)+n)$, incluindo zerar a estrutura. Para D fixo, Θ(n+1), inclusive a chamada vazia da implementação. O espaço é Θ(ceil(D/w)) palavras; com w constante, Θ(D) bits. A notação Θ(1) em n não autoriza chamar 8192 B de memória desprezível.

4. Contagem precisa da força bruta

Na versão que busca apenas entre únicos já emitidos, introduzir os u valores pela primeira vez custa $0+1+\ldots+(u-1)=u(u-1)/2$ comparações, independentemente das buscas de repetidos. Cada uma das n-u repetições custa no máximo u. O máximo para n e u fixos é exatamente $C_{max}=u(u-1)/2+(n-u)u$: introduzir os u valores primeiro e repetir sempre o último do prefixo atinge esse valor. O trabalho total é Θ(n+C), incluindo laço e emissão. Para u=n, $C_{max}=n(n-1)/2$; para u=1, $C_{max}=n-1$.

A aproximação nu/2 para a curva é um cenário de buscas com posições aproximadamente uniformes após estabilizar u, não uma identidade universal. Num cenário de primeira formação dos únicos, há o termo triangular acima. Não se substitui u por n quando n>D. O pior caso limitado ao domínio é O(nD), linear em n se D é constante, embora a constante possa torná-lo impraticável.

5. Fatoriais, subconjuntos e I/O: contagens verificáveis

Bogosort. Há n! permutações de objetos rotulados, mas $\prod c_j!$ delas produzem a sequência de valores ordenada. Portanto $q=\prod c_j!/n!$. Embaralhamentos uniformes independentes dão distribuição geométrica: E[tentativas]=1/q. Para uma entrada inicialmente desordenada, esse é o número esperado de embaralhamentos até o primeiro sucesso; a checagem inicial acrescenta apenas uma verificação. Se já ordenada, a função hipotética termina imediatamente. Tempo esperado cn/q é modelo de custo por tentativa, não garantia. Com n=1000 distintos, $\log_{10}(1/q)=2567{,}604644222$. Para n>D usar n! sem dividir pelas multiplicidades superestima o número de tentativas; usamos lgamma para calcular o logaritmo sem overflow.

Subconjuntos. Cada posição tem duas decisões independentes, logo 2^n folhas e $2^{n+1}-1$ nós na árvore binária completa. Cada posição pertence a exatamente metade dos subconjuntos: $\Sigma_{t=0..n} t\binom{n}{t}=n2^{n-1}$. Com 2 B por elemento, a materialização exige $n2^n$ B só de payload. Assim, materializar e copiar todos os elementos custa $\Theta(n2^n)$ tempo e espaço, além de descritores e pilha Θ(n). Somente enumerar em streaming custa $\Theta(2^n)$ visitas e pode usar heap zero; não confundir as duas variantes. As curvas do relatório usam materialização para AMBOS tempo e RAM. Em n=1000, $\log_{10}(\text{bytes})=304{,}029995664$, e não apenas $\log_{10}(2^{1000})=301{,}029995664$.

Unário. Seja $L_j=x_j+1$ o tamanho em bytes do registro j, com j=1,…,n. Gravar todos os registros custa $S=\Sigma L_j$. Consultar todos os registros anteriores para cada posição, sem parar cedo, relê o registro j exatamente n-j

vezes. O tráfego idealizado total é $V=\Sigma(n-j+1)L_j$. Essa identidade não exige independência. Para comprimentos de mesma esperança $\mu=32768{,}5$, a linearidade dá $E[V]=\mu n(n+1)/2$, e $E[S]=\mu n$. A expressão inclui a gravação inicial e todas as releituras, sem contar novamente um registro corrente que já está disponível no vetor. Para $n=10^7$: $E[V]=1638425163842500000$ B, aproximadamente $1{,}638425164\times10^{18}$ B, e $E[V]/10^9\approx1{,}638\times10^9$ s a 1 GB/s. Não estão incluídos syscalls, seeks, comparar contadores e outros custos; esse throughput é hipótese otimista. O custo completo de CPU inclui $\Omega(n)$ acessos ao vetor e o custo de contar símbolos; no domínio fixo, o modelo de varredura completa tem $\Theta(n^2)$ tempo de I/O em melhor e pior caso, com constantes dependentes dos comprimentos.

6. Probabilidade do Bloom e o significado dos arredondamentos

Com ku marcações uniformes independentes, a probabilidade de um bit particular continuar zero é exatamente (1-1/m)^(ku). Aproximá-la por exp(-ku/m) e supor independência entre os k bits consultados leva a p≈(1-exp(-ku/m))^k. As ocupações dos bits não são rigorosamente independentes; trata-se do modelo clássico aproximado. O modelo de dupla hash, hashes não uniformes e consultas adversariais podem alterar a taxa. Não se apresenta p como limite determinístico garantido.

No ótimo contínuo, k=(m/u)ln2 e m=-u ln p/(ln2)². Porém k é inteiro. Para u=D e p-alvo=0,01, escolher k=7 exige ajustar m≥-7u/ln(1-p^(1/7)). Arredondando para bytes, 78586 B dão p≈0,0099996885 pela aproximação. Usar 78521 B e simplesmente arredondar k para 7 dá p≈0,010039, ligeiramente acima da meta. Essa correção foi aplicada. Como Bloom não foi implementado, não há taxa empírica observada; uma implementação escolhida precisaria medir a taxa e avaliar as hipóteses. Ambas as soluções entregues são exatas.

Uma taxa p por consulta não implica saída inteira correta com probabilidade 1-p: para r consultas inéditas, a desigualdade da união limita a probabilidade de algum falso positivo por $\min(1,\Sigma p_i)$, quando limites marginais válidos estão disponíveis. Os eventos não precisam ser independentes para essa desigualdade. A taxa admissível do PDF não é necessária às nossas implementações exatas.

7. Decisão por restrição e interpretação dos gráficos

Embarca: provar espaço constante não basta; a constante precisa caber em 256 B. Heap Sort tem prova de correção e evidência local de 96 B. Bitmap usa 8192 B de dados, impossíveis sob o teto, mesmo antes dos escalares. Datacenter: bitmap tem tempo linear e domínio conhecido; medições locais apoiam sua recomendação, mas não provam o prazo na máquina externa.

Curvas hipotéticas misturam limites, esperanças e cenários declarados: não são oito benchmarks. As multiplicidades balanceadas de Bogosort são uma escolha pessimista de multiplicidades, não a esperança das entradas uniformes medidas. Tempos nominais a 1 ns/op e I/O a 1 GB/s são condições de modelos; não são constantes inferidas de O. As transformações dos eixos preservam os expoentes astronômicos sem converter previsões em evidência empírica.

As verificações numéricas deste capítulo estão em scripts/validar_matematica.py, com registro em dados/revisao_matematica.txt. Elas conferem identidades e arredondamentos em casos pequenos por contagem independente; não implementam uma terceira solução de deduplicação.

Análise teórica — Duas empresas, duas restrições

Escopo, interpretação e hipóteses

O projeto compara Heap Sort iterativo e bitmap para deduplicação de leituras de 16 bits. As duas implementações utilizam C; Python é empregado na automação dos experimentos e dos documentos. A seção 6 do enunciado permite a escolha da linguagem, enquanto os objetivos e o critério 3 da rubrica mencionam Pascal. A confirmação da linguagem e a aprovação do par são requisitos acadêmicos registrados na checklist.

Seja n o número de leituras, u o número de valores distintos e D=65536. Temos $0 \le u \le \min(n,D)$. Para n=100000 e n=10000000, duplicatas são inevitáveis. O vetor usa 2n bytes com uint16_t; n=10000000 ocupa 20000000 bytes. O limite Embarca é de 256 bytes EXTRAS ao vetor, mesmo que esse vetor não caiba num sensor real; essa residência é premissa do enunciado. A Datacenter exige tempo estritamente menor que 2 s na máquina da disciplina. A Embarca não tem prazo numérico explícito. Não atribuímos a ela o limite de 2 s.

Modelo RAM: leitura, comparação, acesso por índice e operação de palavra de 64 bits custam O(1); índices cabem em size_t. Nos limites físicos reais não se considera n além da memória endereçável. As análises genéricas permitem D crescer; com D fixo, muitas expressões em u se tornam lineares em n com constantes grandes. Pilha, heap e disco serão separados. O espaço de saída está no prefixo do vetor sempre que possível, sem cobrar outro vetor de saída.

1. Força bruta

Para cada leitura, procurar no prefixo de únicos; se não existir, anexar nesse prefixo. A escrita fica atrás da leitura, portanto é exata e preserva primeiras ocorrências. O trabalho é a soma das posições percorridas nas buscas. Melhor caso $\Theta(n)$, com todos iguais. Pior genérico $\Theta(n^2)$, pois $0+1+\ldots+(n-1)=n(n-1)/2$ quando distintos. Para u limitado, limite $O(nu)$; configurações que mantêm muitas buscas longas atingem $\Theta(nu)$. No modelo médio de posições equiprováveis, cerca de nu/2 comparações após formar o conjunto. Sem distribuição não há uma média universal. Com D fixo, pior $O(nD)$, não crescimento quadrático indefinido.

Pilha $\Theta(1)$, heap 0, disco 0. Pequeno número de índices pode caber nos 256 B, mas qualquer implementação exigiria verificação da ABI. Embarca: candidata, porém escolhida a ordenação para evitar até 65536 buscas por leitura. Datacenter: rejeitada como recomendação para lote geral. Em $n=10^7$ e $u \approx D$, $nu/2 \approx 3{,}28 \times 10^{11}$ comparações; a 1 ns/comparação seriam 328 s. Isso é hipótese, não benchmark. Todos iguais podem ser rápidos. A observação de que seria o único candidato seguro é excessiva: Heap Sort iterativo também tem espaço constante.

2. Ordenação e varredura

Ordenar agrupa iguais; manter o primeiro de cada bloco e compactar. O(n) da varredura soma-se ao custo da ordenação. Na implementação, construir heap custa $\Theta(n)$: a soma de alturas é $O(n)$, pois há menos nós em níveis altos. Cada uma de n-1 extrações pode percorrer $\log_2 n$ níveis: $O(n \log n)$. Para chaves distintas e ordem não favorável, tempo médio e pior $\Theta(n \log n)$; com todos iguais, o sift encerra imediatamente e o melhor é $\Theta(n)$. O limite superior continua válido com repetições; não se deve prometer $\Theta(n \log n)$ em toda entrada.

Heap Sort iterativo usa pilha $\Theta(1)$, heap 0, disco 0, altera ordem para crescente. A prova de corretude combina: sift restaura o heap; a raiz máxima vai à extremidade definitiva; após ordenar, valores diferentes do último emitido são exatamente os únicos. Quick Sort usual tem média $\Theta(n \log n)$, pior $\Theta(n^2)$, pilha média $\Theta(\log n)$ e pior $\Theta(n)$, sem garantir 256 B. Merge Sort usual tem $\Theta(n \log n)$ em todos os casos e vetor auxiliar $\Theta(n)$, além de pilha $\Theta(\log n)$. Não são implementados aqui. Embarca: Heap Sort atende espaço neste build; Datacenter: possível, mas trabalho e acessos são maiores que os do bitmap, sem benefício exigido pelo problema.

3. Tabela hash/conjunto

Consultar cada x; inserir e emitir apenas quando ausente. Com fator de carga controlado, boa dispersão e hash universal/randomizado, consultas esperadas O(1) e inserções amortizadas O(1), total esperado $\Theta(n)$; melhor $\Theta(n)$. Redimensionar ocasiona operação individual $\Theta(u)$, coberta pela análise amortizada. Colisões adversas com listas ou sondagem podem levar a $O(nu)$, $O(n^2)$ genérico; não se supõe árvore nos buckets. Domínio fixo dá $O(nD)$ nesse pior caso.

Heap $\Theta(u)$ entradas (incluindo buckets e fator de carga), pilha $\Theta(1)$ na versão iterativa, disco 0. Modelo dos gráficos: 16 B por distinto, hipótese otimista de chave, estado, alinhamento e folga; bibliotecas podem usar

mais. Com D cheio, cerca de 1 MiB, contra 8 KiB do bitmap. Embarca: não garante 256 B para todos n/u. Datacenter: plausível, mas não há razão para aceitar colisões, alocações e overhead neste domínio. Para u muito pequeno cabe um conjunto diminuto; isso não garante o limite geral.

4. Árvore balanceada (AVL/Red-Black)

Buscar x e inserir somente se ausente; emitir no prefixo ou percorrer em ordem. Altura O(log(u+1)), pelo invariante de balanceamento; logo O(n log(u+1)+u) no limite superior. Com u≈n e chaves diversas, médio/pior Θ(n log n) genérico. Melhor Θ(n) com todos iguais; uma entrada com muitos acertos na raiz também pode custar Θ(n). Não se afirma limite inferior log u para toda distribuição. Rotações custam O(1) por ajuste; manutenção do caminho cabe no limite da altura.

Heap Θ(u) nós; modelo de 32 B/nó para dois ponteiros, pai/balanceamento, chave e alinhamento, cerca de 2 MiB no domínio completo. Pilha Θ(log u) se recursiva; iterativa pode usar pai e Θ(1) de pilha. Disco 0. Embarca: nós e possível pilha violam a garantia para u grande. Datacenter: determinística e exata, mas perseguição de ponteiros e log u são desnecessários frente ao acesso direto. Quando se precisa de consultas ordenadas dinâmicas em domínio amplo, ela tem utilidade.

5. Bitmap

Associar um bit a cada valor. Inicializar D bits em zero, ler cada x, testar bit e emitir/marcar somente se zero. No código, cada palavra tem 64 bits; índice x>>6 e máscara 1<<(x&63). Invariante: o bit representa exatamente os valores já visitados. Como u≤i+1, compactar não sobrescreve elementos futuros. Retorna únicos na ordem das primeiras ocorrências.

Tempo Θ(n+D/w), w=64, nos três casos, pois sempre inicializa 1024 palavras e lê n elementos. Com D fixo, Θ(n+1); o atalho usual O(n) pressupõe n≥1. A variante que primeiro marca todos e depois percorre o domínio devolve ordem crescente em Θ(n+D), mas NÃO é o código entregue. Espaço D/8=8192 B mais escalares: aqui automático, na pilha; heap 0 e disco 0. Ser constante em n não significa caber em 256 B. Embarca: rejeitado, 32 vezes o orçamento só em bits. Datacenter: escolha exata, linear, sem colisões, com conjunto de trabalho pequeno. Os dados locais apoiam a escolha, mas a máquina de referência ainda precisa ser testada.

6. Filtro de Bloom

m bits, k funções de hash; um valor é considerado presente se todos seus k bits estiverem marcados. Se algum estiver zero, emitir e marcar. Após u inserções, com hashes uniformes independentes, $p \approx (1-\exp(-ku/m))^k$. O ótimo $k \approx (m/u)\ln 2$, $m \approx -u \ln p/(\ln 2)^2$. Para u=65536 e p=0,01: m≈628167 bits (arredondar a bytes), cerca de 78521 B no ótimo contínuo. Com k inteiro igual a 7, deve-se aumentar para 628684 bits, arredondados a 78586 B (628688 bits), para p≈0,99997% na fórmula aproximada. A fórmula de dimensionamento com k fixo é $m \geq -ku/\ln(1-p^{(1/k)})$; arredondar k sem ajustar m pode passar ligeiramente de 1%. Para u=1000: cerca de 1199 B, ainda acima de 256 B. Dimensionar por $n=10^7$ desperdiçaria memória: só existem D chaves. Com m=2048 bits e u=D, k ótimo seria menor que 1; k=1 já dá p≈1-exp(-32), quase 100%.

Melhor/médio/pior Θ(nk+m/w) numa variante que avalia sempre os k hashes; com interrupção antecipada o custo varia entre Ω(n+m/w) e O(nk+m/w). Para k fixo é linear. Pilha Θ(1) escalares se bitset no heap, heap Θ(m) bits, disco 0; um bitset automático transfere esse custo para a pilha. Falso positivo significa descartar uma leitura inédita e perder um valor na saída; não é duplicata remanescente. Não há falso negativo na consulta ao filtro para valores inseridos sob essas hipóteses. p por consulta inédita não é chance de saída inteira correta, nem um limite determinístico em toda sequência. O PDF admite probabilístico com taxa declarada ≤1%; aqui preferimos exatidão. Embarca não comporta o dimensionamento; Datacenter pode aceitar aproximação em outros problemas, mas neste domínio o bitmap é menor e exato. Não implementado.

7. Bogosort e varredura

Checar se está ordenado; se não, embaralhar uniformemente (Fisher-Yates in-place) e repetir. Depois compactar vizinhos. Para multiplicidades $c_1,\ldots,c_u$, a probabilidade de uma permutação de elementos rotulados estar não decrescente é $q=\prod c_j!/n!$. O número esperado de tentativas geométricas é $1/q=n!/\prod c_j!$. Com distintos, n!; com todos iguais, 1. Se embaralhar e checar custam $\Theta(n)$, tempo esperado $O(n/q)$; se já ordenado, melhor $\Theta(n)$. Pior caso não tem teto finito: o processo pode continuar indefinidamente. Para modelo médio é preciso declarar a distribuição das multiplicidades e a aleatoriedade dos embaralhamentos.

Pilha $\Theta(1)$, heap 0 e disco 0 se embaralhamento e gerador forem iterativos de estado fixo. RNG e ABI devem caber no orçamento, não basta dizer O(1). Sem garantia temporal; não há orçamento explícito Embarca, mas o tempo geral é impraticável. Datacenter não tem garantia de 2 s. Domínio fixo não salva o método: com frequências balanceadas, o multinomial continua crescendo aproximadamente como D^n dividido por fator polinomial para D fixo. Casos ordenados ou minúsculos podem terminar rápido.

8. Enumeração recursiva de subconjuntos

Para tornar a estratégia definida: decidir incluir/excluir cada posição, percorrer as 2^n folhas e examinar os subconjuntos de cardinalidade 1 para recuperar seus elementos, emitindo apenas primeiras ocorrências por uma busca no prefixo de únicos. É um modo correto, mas extremamente redundante, de resolver um problema mais geral. Identidade de posições evita ambiguidade quando valores se repetem. Nada exige que subconjuntos de valores coincidentes sejam armazenados uma única vez.

A árvore tem 2^{n+1}-1 nós: visitar sem copiar é $\Theta(2^n)$ tempo em melhor/médio/pior para essa variante que não poda nem encerra cedo, mais O(nu) para as emissões de singletons. Recursão tem profundidade n+1 e pilha $\Theta(n)$ frames; uma seleção corrente ocupa O(n) se não couber no próprio caminho. Heap pode ser 0 em enumeração por streaming, portanto $\Theta(2^n)$ de heap não é obrigatório. Materializar listas de todos os subconjuntos exige $\Theta(n2^n)$ tempo e bytes de elementos: cada uma das n posições participa de 2^{n-1} subconjuntos. Acrescentam-se até $\Theta(2^n)$ descritores. Copiar/examinar cada folha inteira também custa $\Theta(n2^n)$. Disco 0, salvo armazenamento externo que transfira o mesmo volume. Embarca: pilha viola 256 B em n grande, mesmo sem materializar; Datacenter: explosão de tempo. É útil para busca combinatória com n pequeno, não para deduplicação em lote.

9. Codificação unária em disco

A tabela do PDF descreve serialização, mas não especifica como comparar e remover duplicatas. Apenas escrever os valores não resolve o problema. Definimos uma variante exata e de RAM constante para análise: gravar cada leitura x como x bytes '1' seguidos de um byte '0'; para cada registro, comparar seu comprimento com os registros anteriores, emitindo o x original somente se não houver igual. Comparações e navegação usam contadores de largura fixa e I/O sem buffer grande; o disco continua contendo todos os registros, inclusive os repetidos. A saída é compactada no vetor de entrada. Essa variante não é implementada.

Volume $S=\Sigma(x_i+1)$, entre n e 65536n bytes; zero precisa do delimitador. Só codificar custa $\Theta(n+\Sigma x_i)$, NÃO tempo exponencial em n para domínio fixo. Comparar por varredura física dos registros anteriores pode custar $O(nS)$, com $\Theta(n^2D)$ no cenário de comprimentos longos e busca que não encerra cedo. Média depende de ordem e multiplicidades; modelo pessimista dos gráficos lê metade do arquivo para cada consulta, sem atalhos, mesmo após encontrar duplicata. O tráfego exato dessa auditoria é $V=\Sigma(j=1..n)(n-j+1)(x_j+1)$, independentemente de duplicatas; $\Theta(nS)$ é atingível, mas não é identidade para comprimentos arbitrários quando D também varia. Melhor $\Theta(n^2)$ quando todos os comprimentos são 1 nesse modelo; com D fixo todos esses limites são quadráticos em n, com constantes distintas. Uma versão que encerra cedo seria mais rápida. Não atribuímos $\Theta(S)$ ao deduplicador completo.

Pilha/heap de trabalho podem ser constantes e inferiores a 256 B numa implementação de baixo nível com buffers mínimos, mas libc, FILE e buffers do kernel precisam ser contabilizados conforme a plataforma. Disco $\Theta(S)$. Não há medição nem garantia de RAM para essa estratégia hipotética. Se um valor tem b bits e

chega a 2^b-1, representá-lo em unário exige $\Theta(2^b)$ símbolos; é exponencial em b, não em n quando b=16 fixo. Codificar bits em vez de bytes reduz volume até 8 vezes, mantendo necessidade de delimitadores e a análise de crescimento. Embarca pode falhar no armazenamento, disponibilidade de disco, buffers e tempo prático; não existe restrição numérica de disco no enunciado, portanto o volume de disco caracteriza uma limitação prática, sem teto formal de capacidade especificado. Datacenter: o modelo de I/O abaixo excede 2 s. Unário serve em exemplos minúsculos, contagens pequenas e formatos didáticos.

10. Matriz n×n de auditoria

Construir $M_{ij}=(a_i=a_j)$ para todos os pares; uma posição i é primeira ocorrência se nenhum j<i tem igualdade. A construção é $\Theta(n^2)$ em melhor/médio/pior mesmo com todos iguais, pois a variante exige auditar todos os pares. A leitura e compactação adicionais não dominam. Usar somente metade altera constante, não ordem. NaN não existe no domínio inteiro, logo igualdade é bem definida.

Heap $\Theta(n^2)$ células, pilha $\Theta(1)$, disco 0 na versão residente; em disco há o mesmo crescimento em outro recurso. Em bytes n^2 B; empacotada ceil(n^2/8) B, além de escalares. Embarca: mesmo n=1000 usa 125000 B em bits, muito além de 256 B. Datacenter: sem teto de memória formal, mas exigir 12,5 TB ou 100 TB em $n=10^7$ é operacionalmente oneroso; tempo também é impeditivo sob o modelo quantitativo. Não afirmar que viola só memória. Matriz pode servir a uma auditoria exigida para n pequeno; auditoria de todos os pares não foi pedida aqui.

Exemplos quantitativos das quatro estratégias exageradas

Bogosort: n=1000 com todos distintos é válido no domínio. $\log_{10}(1000!)\approx2567{,}604644$; cerca de $4{,}02\times10^{2567}$ tentativas. Supondo, de modo extremamente favorável, 1 ns por elemento para embaralhar e verificar, cada tentativa custa 10^{-6} s: tempo $\approx4{,}02\times10^{2561}$ s. Todos iguais têm q=1, logo esse número não vale para qualquer entrada. Para n>D usa-se o multinomial das multiplicidades, jamais n! sem correção.

Power set: n=1000 implica $2^{1000}\approx1{,}072\times10^{301}$ subconjuntos. Materializar elementos uint16_t custa $2\times1000\times2^{999}=1000\times2^{1000}\approx1{,}072\times10^{304}$ B, sem contar descritores. Mesmo uma representação otimista de 16 B/frame já usa 16016 B para profundidade 1001, violando Embarca. A 10^9 folhas/s, apenas enumerar requer cerca de $1{,}072\times10^{292}$ s. Em $n=10^7$ a pilha desse modelo seria aproximadamente 160 MB antes da materialização. Para n≤10, enumeração pode ser razoável em outro problema.

Unário: com $n=10^7$ e média uniforme E[x]=32767,5, S esperado=327685000000 B (327,685 GB decimais). Só gravar a 1 GB/s sustentado custa 327,685 s; ler novamente soma esse custo. Pior S=655360000000 B, 655,36 GB. No deduplicador definido, tráfego esperado do modelo $32768{,}5\times n(n+1)/2\approx1{,}638425164\times10^{18}$ B, ou $1{,}638\times10^9$ s à mesma taxa otimista, cerca de 51,9 anos. São hipóteses de throughput, sem latência de seek ou CPU; não medições. Um buffer pequeno evita grande RAM de usuário, mas não elimina disco/I/O. Em n=1, x=0, apenas um byte basta.

Matriz: $n=10^7$ produz 10^{14} células: 100 TB em bytes ou 12,5 TB em bits (≈11,37 TiB). A 1 ns/célula, construção idealizada leva 100000 s (27,78 h), sem alocação nem tráfego de memória. n=1000 já pede 1 MB ou 125 kB; mesmo n=100 pede 1250 B em bits. n=10 pede 13 B em bits mais escalares e pode caber. Só para Datacenter existe teto temporal formal de 2 s; rejeição temporal da Embarca é julgamento de viabilidade prática.

Escolha e contrato

Implementamos exatamente duas estratégias do catálogo: ordenação Heap Sort seguida de compactação e bitmap. A infraestrutura de testes usa histograma de frequências exclusivamente como oráculo, não como terceira solução entregue ao cliente. A geração, validação, cópia e impressão estão fora do tempo do algoritmo.

Ambas recebem uint16_t *a e size_t n, permitem NULL somente se n=0, alteram a e retornam size_t u. Apenas a[0..u) é saída; a cauda não tem contrato. Heap devolve crescente; bitmap mantém primeiras

ocorrências. Ambas são determinísticas e exatas. Nenhuma exige estabilidade no PDF; a diferença está documentada e validada.

Heap supera força bruta em previsibilidade de trabalho para u alto, não exige estruturas do hash/árvore, nem bits além de 256 B. A pilha de Quick Sort recursivo não foi presumida segura. Bitmap aproveita o domínio: custa menos que Bloom a 1%, evita colisões de hash e ponteiros de árvore. A compactação integrada evita uma segunda varredura por todo o vetor ou domínio para emissão.

Referências verificáveis

- Cormen, Leiserson, Rivest e Stein. Introduction to Algorithms, 4ª edição. MIT Press, 2022. Heap Sort, tabelas hash e árvores balanceadas. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/

- Bloom, Burton H. Space/Time Trade-offs in Hash Coding with Allowable Errors. Communications of the ACM, 13(7), 1970, pp. 422–426. https://doi.org/10.1145/362686.362692

- Broder, A.; Mitzenmacher, M. Network Applications of Bloom Filters: A Survey. Internet Mathematics 1(4), 2004, pp. 485–509. https://doi.org/10.1080/15427951.2004.10129096

- GCC, Developer Options: -fstack-usage. https://gcc.gnu.org/onlinedocs/gcc/Developer-Options.html

- POSIX, clock_gettime. https://pubs.opengroup.org/onlinepubs/9799919799/functions/clock_gettime.html

- Linux man-pages, getrusage(2), ru_maxrss em KiB no Linux. https://man7.org/linux/man-pages/man2/getrusage.2.html

Metodologia e evidências de memória

Compilação C11 com GCC, -O3, -Wall -Wextra -Wpedantic -Werror. Para tornar os frames auditáveis: -fno-inline -fno-omit-frame-pointer -mno-red-zone -fno-tree-loop-distribute-patterns -fstack-usage. Flags completas e hardware reportado por lscpu estão em dados/ambiente.json. São flags para GCC/x86-64, não uma configuração portátil de microcontrolador. Não se utilizou -march=native. A comparação é entre builds de mesmas opções, não entre bibliotecas com otimizações distintas.

Cada combinação algoritmo/n roda em um processo separado. Entradas uniformes aproximadas são os 16 bits baixos do xorshift32 de semente decimal 20261108, reiniciada para cada processo. O gerador não é criptográfico. Mesmos dados são usados nos dois algoritmos. Executa-se um aquecimento e sete rodadas medidas; o vetor original é copiado antes de cada rodada. O relógio CLOCK_MONOTONIC fornece tempo de parede com nanossegundos na representação; isso não promete precisão de 1 ns. A janela inclui chamada e retorno da deduplicação, construção do heap ou limpeza do bitmap e compactação. Exclui malloc dos vetores, geração, memcpy, oráculo e printf. Para n pequeno o overhead do relógio é relevante e não foi artificialmente subtraído. Estatística: mediana; dispersão por mínimo/máximo e quartis exclusivos de statistics.quantiles. Não são intervalos de confiança.

Corretude: testes manuais, domínio completo crescente e decrescente, 200 casos pseudoaleatórios menores, e quatro distribuições em cada escala obrigatória (iguais, ciclos crescentes, ciclos decrescentes, pseudoaleatória). São 222 casos para cada algoritmo, além das chamadas NULL/0. O histograma de teste verifica existência de cada saída, ausência de repetição, contagem exata e ordem contratada. Validação também é feita após cada rodada do benchmark. Asserções permanecem ativas, sem -DNDEBUG. AddressSanitizer e UndefinedBehaviorSanitizer são executados em build separado: seus tempos e frames não são usados como benchmark.

A memória de trabalho foi auditada por -fstack-usage nos objetos realmente usados na medição, não pela soma de sizeof de variáveis. Frames reportados neste GCC: heap_unique 48 B e sift 32 B, simultâneos no caminho mais profundo: 80 B. Adiciona-se margem conservadora de 16 B de fronteira de chamada/ABI, total 96 B. bitmap_unique tem frame de 8208 B, mais a mesma margem: 8224 B. Os frames incluem ponteiros/argumentos salvos, temporários, alinhamento e salvamento de registradores conforme gerados pelo compilador. Arquivos .su são preservados em dados/evidencias/; desmontagem está em

dados/desmontagem.txt. O compilador classifica todos os frames como static. Não há recursão nem chamadas de sift para outras funções. nm -u confirma ausência de símbolos externos nos dois objetos; inspeção da fonte e desmontagem confirma heap dinâmico de 0 B. Esse zero é evidência estática do binário, não amostragem de RSS nem profiler de malloc.

O limite de 96 B independe de n, incluindo os três tamanhos testados. A verificação automatizada falha se a soma mais margem superar 256 B. Isso demonstra a restrição no build local x86-64, mas não em qualquer ABI ou compilador. Interrupções, pilha do sistema operacional, rotina chamadora/gerador e redirecionamentos de runtime não fazem parte da subárvore do algoritmo. Em firmware real, medir novamente com o compilador do alvo e reservar a pilha de interrupções conforme requisitos do dispositivo. Com sanitizadores ou instrumentação diferente, os frames mudam.

O bitmap foi posto na pilha por simplicidade e reentrância; 8 KiB são adequados ao ambiente local e proibidos para Embarca. Transferi-lo para heap não reduziria memória EXTRA. O vetor original e sua cópia usam 4n B na infraestrutura; o cliente precisa apenas do vetor de 2n B. O oráculo usa histograma e auxiliares exclusivamente nos testes. getrusage fornece ru_maxrss em KiB no Linux, pico do processo incluindo essa infraestrutura; não é o consumo instantâneo do algoritmo. Dados calculados (2n, D/8), reportados pelo compilador (frames) e observados pelo SO (RSS) são identificados separadamente.

Resultados locais efetivamente medidos

CPU informada ao ambiente: AMD EPYC 9V74 80-Core Processor. Sistema: Linux-6.18.57-x86_64-with-glibc2.41. Compilador: gcc (Debian 14.2.0-19) 14.2.0.

Algoritmo	n	u	Mediana (s)	Mín–máx (s)	Q1–Q3 (s)	Extra conservador (B)	RSS pico (KiB)
heap	1000	996	0.000015543	0.000014461–0.000045778	0.000014943–0.000027561	96	11904
bitmap	1000	996	0.000001002	0.000000991–0.000001062	0.000000991–0.000001012	8224	11904
heap	100000	51485	0.008639077	0.008214097–0.010335410	0.008412892–0.009500212	96	11904
bitmap	100000	51485	0.000325342	0.000308507–0.001213317	0.000309929–0.000426512	8224	11904
heap	10000000	65536	1.623778035	1.498388893–1.828633656	1.563083458–1.781125109	96	40192
bitmap	10000000	65536	0.006033297	0.006005024–0.006242396	0.006009632–0.006099775	8224	40192

O RSS é o máximo residente de cada processo Linux, inclui dois vetores, oráculo, runtime e páginas tocadas. Não é memória extra do algoritmo e não deve ser subtraído do vetor para inferir a pilha. A inicialização do bitmap está dentro do cronômetro.

Extrapolações: hipóteses, unidades e limites

As oito estratégias não foram programadas nem cronometradas. Seus pontos são previsões ilustrativas, não afirmações de que executariam nesses tempos. Para tempo, usa-se custo nominal $c=10^{-9}$ s por operação abstrata; não há calibração independente dessas operações. Um hash ou nó de árvore é modelado com 5 operações e Bloom com 7 hashes de custo c, escolha didática otimista. Não se derivou c de Big-O nem se ajustaram as curvas para garantir uma classificação. Multiplicar essas constantes por 0,1 ou 10 desloca os $\log_{10}$ dos tempos em -1 ou +1, sem remover a explosão combinatória. Tráfego unário supõe 10^9 B/s. Cache, branches, largura de banda e latência estão ausentes desses modelos; a incerteza é grande e não é expressa como erro estatístico.

Nas curvas força bruta/hash/árvore/Bloom, $u(n)=D[1-(1-1/D)^n]$, esperança sob amostragem uniforme independente. O gerador real é determinístico e só aproxima essa hipótese. Os u reais são registrados, não substituídos pela esperança. Para Bogosort, as multiplicidades são balanceadas: q=floor(n/D), r=n mod D,

com D-r contagens q e r contagens q+1. Quando $n \leq D$, isso é uma entrada distinta (pior cenário de multiplicidades), diferente da distribuição empírica; não se pretende que todas as curvas sejam o tempo da mesma instância. Usar n! em n>D seria incorreto. O modelo de subconjuntos materializa todos os elementos para tempo e memória, com fator n em ambas as curvas. A variante de streaming é discutida separadamente na análise.

Modelos de tempo em segundos:

- Força bruta: c n max(1,u/2), busca média no prefixo de únicos.

- Hash: 5cn, esperança com boa dispersão; não descreve colisões adversas.

- Árvore: $5cn \log_2(u+1)$, limite de buscas em árvore balanceada.

- Bloom: 7cn + 78586c, limpeza conservadora por byte do filtro máximo mais hashes.

- Bogosort: cn n!/[(q!)^(D-r)((q+1)!)^r], checagem/embaralhamento nominal de cn.

- Subconjuntos: $n2^n c$, custo nominal proporcional aos elementos materializados (um fator constante 2 é absorvido por c).

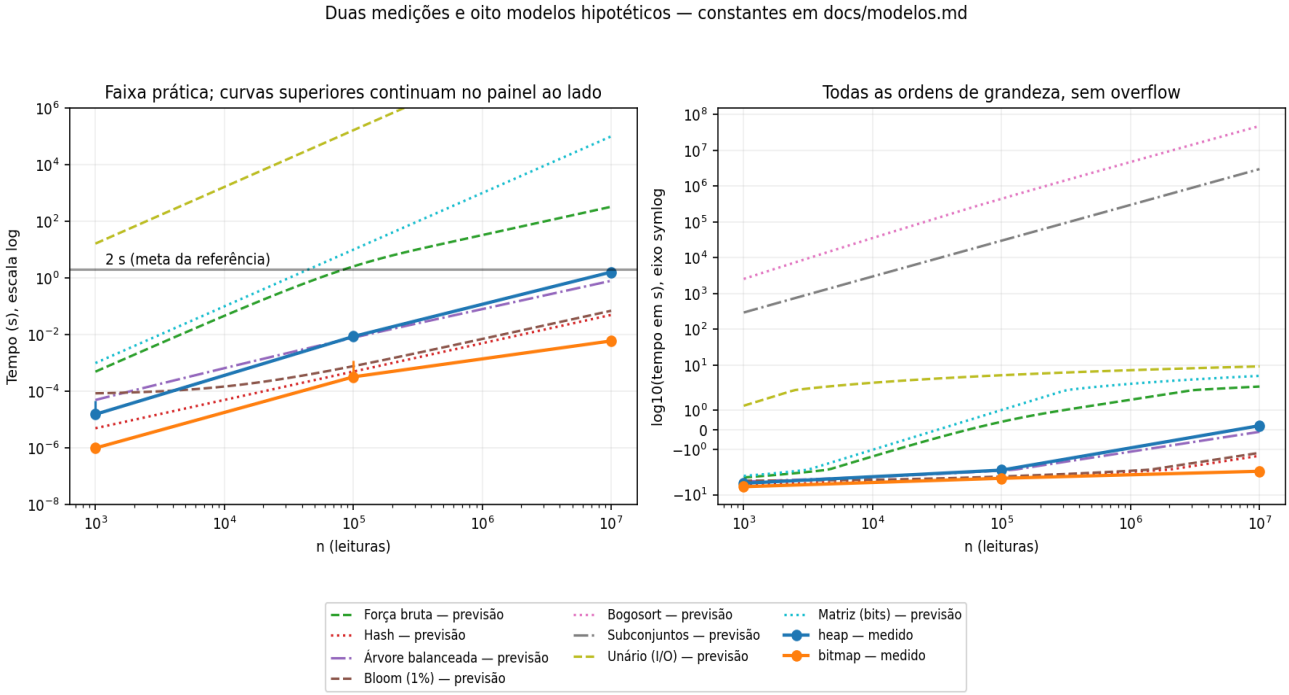
- Unário: $[32768{,}5 \times n \times (n+1)/2]/10^9$, gravar e varrer em média metade do arquivo por consulta; sem seeks e com comprimentos médios uniformes.

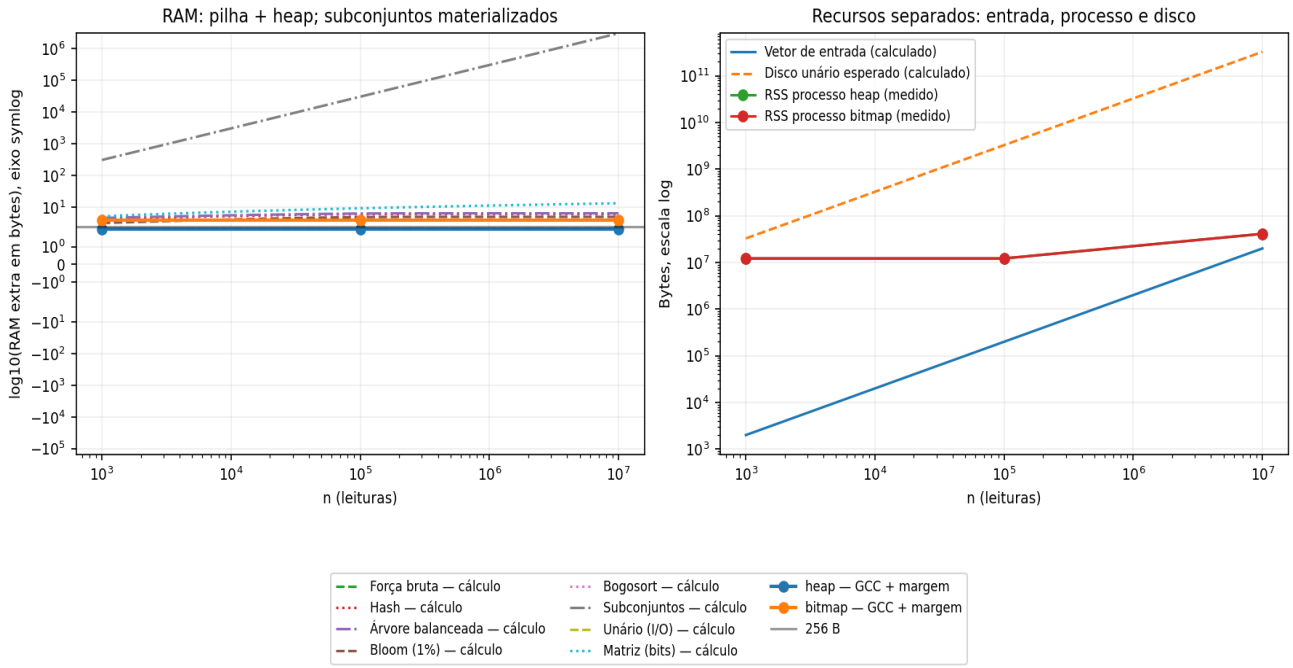
- Matriz: cn^2, uma operação nominal por célula; não inclui alocação.

RAM extra hipotética em bytes (não medidas): força bruta/Bogosort/unário 128 B de trabalho nominal; hash 16u+128; árvore 32u+128; Bloom ceil[-7u/ln(1-0,01^(1/7))/8]+128, com k=7 e correção discreta para a meta de 1%; subconjuntos $n2^n$ (elementos uint16_t, sem descritores, além da pilha $\Theta(n)$); matriz $\text{ceil}(n^2/8)+128$, empacotada. Para subconjuntos, o volume materializado domina a pilha nas escalas dos gráficos; não se deve interpretar isso como memória obrigatória de streaming. Unário usa disco separado $n \times 32768{,}5$ B esperado. Os 128 B são hipótese ilustrativa de escalares e buffer, não garantia de ABI. Comparações no relatório consideram também matrizes em bytes.

A fonte scripts/documentos.py usa lgamma para fatoriais e expoentes em $\log_{10}$ para 2^n. dados/previsoes_log10.csv contém as oito curvas sem overflow e com unidade explícita. Em graficos/tempo.png, o painel esquerdo mostra segundos em escala log de 10^{-8} a 10^6; as curvas fora dessa janela são continuadas integralmente no painel direito. O painel direito mostra o número $\log_{10}$(segundos) sobre eixo symlog: valor 6 representa um milhão de segundos, valor 3000000 representa 10 elevado a três milhões de segundos, não três milhões de segundos. Essa segunda transformação permite ver simultaneamente números negativos (tempos <1 s), valores próximos de zero e expoentes enormes. Nenhuma curva inviável é removida.

As duas linhas sólidas com pontos são medianas medidas; barras no painel prático são mínimo/máximo. As outras oito linhas são modelos. graficos/memoria.png usa o mesmo tratamento de expoentes para RAM e outro painel em bytes para disco, entrada e RSS. O frame GCC mais a margem é constante para os dois algoritmos, e não uma medição dinâmica de pico a cada n. Sobreposições em curvas de RAM constante são esperadas; legenda e CSV permitem identificá-las.


Duas medições e oito modelos hipotéticos — constantes em docs/modelos.md
Faixa prática; curvas superiores continuam no painel ao lado
Todas as ordens de grandeza, sem overflow
Tempo (s), escala log
log10(tempo em s), eixo symlog
n (leituras)
n (leituras)
2 s (meta da referência)
Força bruta — previsão
Hash — previsão
Árvore balanceada — previsão
Bloom (1%) — previsão
Bogosort — previsão
Subconjuntos — previsão
Unário (I/O) — previsão
Matriz (bits) — previsão
heap — medido
bitmap — medido



Discussão crítica e recomendação

Os resultados locais indicam que Heap Sort atende ao orçamento de memória com 96 B conservadores e bitmap oferece forte vantagem de tempo no maior lote. O bitmap tem 8192 B só de bits e não serve à Embarca. Para Datacenter, recomenda-se bitmap, condicionado à repetição na máquina da disciplina. Um resultado local menor que 2 s não cumpre sozinho a validação exigida na referência.

A taxa por elemento do bitmap não é constante entre as escalas: enquanto ainda surgem muitos inéditos, a função realiza mais escritas, decisões e atualizações de bits. Em $n=10^7$, há no máximo 65536 emissões; a maior parte das leituras apenas consulta bits. Já o heap percorre caminhos e faz trocas em regiões cada vez maiores do vetor. O vetor de 20 MB muda a localidade frente aos 2 kB da menor escala, enquanto o bitmap mantém somente 8 kB de bits. Esses mecanismos são compatíveis com as diferenças observadas, mas não foram isolados por contadores de cache ou um experimento causal. Não atribuímos uma diferença especificamente a cache, compilador ou SO sem medi-los.

Há efeitos de aquecimento, escalonamento, compartilhamento da CPU virtual, frequência e resolução efetiva do relógio. As sete rodadas e dispersões documentam variação local; não eliminam essas fontes. A cópia anterior aquece páginas e faz parte da condição experimental, embora esteja fora da janela de tempo. O oráculo também toca dados entre rodadas. Comparar tempos de modelos a 1 ns/op com esses pontos não é medir as oito alternativas. A teoria oferece tendências e limites, e os benchmarks oferecem evidências apenas das duas implementações.

O Heap Sort também pode atender ao prazo local, mas isso não o torna a melhor recomendação Datacenter: bitmap faz menos trabalho e explora uma garantia do domínio. Se o domínio crescer ou a estabilidade for obrigatória para o cliente embarcado, a escolha precisará ser revista. Se surgir requisito de manter a entrada intacta, ambos os contratos atuais deixam de atendê-lo sem cópia externa.

Pendências reais: aprovação do par, confirmação de C diante da rubrica que menciona Pascal e reprodução no hardware da disciplina. A checagem de pilha precisa ser refeita em qualquer novo alvo.

Checklist de entrega e validação

Legenda: CONCLUÍDO significa produzido e/ou verificado localmente; PENDENTE é uma exigência externa ainda não demonstrada. A checklist não equivale a nota atribuída.

Requisito	Situação e evidência
Interpretação do enunciado	CONCLUÍDO; docs/analise_teorica.md
Domínio 0–65535 e n=10^3,10^5,10^7	CONCLUÍDO; uint16_t, CSV real e casos nas três escalas
Provas, somatórios e probabilidades auditadas	CONCLUÍDO; fundamentacao_matematica.md e 105 verificações independentes em revisao_matematica.txt
Dez análises com tempo, espaço e hipóteses	CONCLUÍDO; dez seções em analise_teorica.md
Exatamente duas estratégias implementadas	CONCLUÍDO; src/heap.c e src/bitmap.c; oráculo apenas em tests
Contratos, ordem e compactação	CONCLUÍDO; dedup.h, código e testes
Heap iterativo sem heap dinâmico ou recursão	CONCLUÍDO; fonte, nm, desmontagem e frames static
Embarca ≤256 B extras independente de n	CONCLUÍDO no build local: 96 B conservadores; refazer em novo alvo
Datacenter <2 s em n=10^7	CONCLUÍDO localmente; PENDENTE na máquina de referência
Casos vazios, unitário, iguais, distintos, ordenados, reversos, intercalados, extremos, pequenos	CONCLUÍDO; tests/test.c e dados/testes.txt
Corretude nas três escalas e comparação exata de conjuntos/ordem	CONCLUÍDO; 222 casos/algoritmo e validação em cada rodada
Compilação e análise de erros de memória	CONCLUÍDO; GCC sem avisos, ASan/UBSan em dados/sanitizers.txt
Benchmark com aquecimento, sete rodadas e dispersão	CONCLUÍDO; CSV bruto 42 linhas e resumo seis linhas
Hardware, SO, compilador, flags, semente e distribuição	CONCLUÍDO; ambiente.json e CSV
Separar algoritmo, entrada e processo; pilha, heap e disco	CONCLUÍDO; metodologia, memória e dois painéis gráficos
Oito extrapolações no gráfico com duas medições	CONCLUÍDO; tempo.png e previsoes_log10.csv
Constantes, hipóteses e incerteza explícitas	CONCLUÍDO; modelos.md
Fatoriais/exponenciais sem overflow ou sumiço	CONCLUÍDO; lgamma e CSV $\log_{10}$, painel de faixa completa
Cálculos das quatro estratégias exageradas	CONCLUÍDO; exemplos numéricos e cenários especiais
Corrigir unário, power set, Bogosort repetido e Bloom	CONCLUÍDO; análise e modelos
Relatório PDF e fonte editável	CONCLUÍDO; relatorio.pdf e docs/relatorio.md
Código, automação, dependências, README, gráficos e ZIP	CONCLUÍDO na entrega; roteiro de reprodução no README
Aprovação prévia do par pelo professor	PENDENTE; não foi informada
Confirmação de linguagem C	PENDENTE; divergência textual com Pascal
Integrantes	Henry Rodrigues, Rafael Maluf e Matheus
Bloom empírico ≤1%	NÃO SE APLICA; Bloom só analisado, soluções exatas

Revisão pelos seis critérios da rubrica

Critério	Peso	Material para revisão e limite
Análise das dez estratégias	25%	analise_teorica.md; inclui deduções, hipóteses, domínio limitado e recursos separados
Justificativa do par	10%	seção Escolha e contrato, comparação com as outras oito; aprovação pendente
Implementação e instrumentação em Pascal	20%	fontes C, testes, frames, desmontagem e sanitizadores; C precisa ser aceito pelo professor
Dados e coerência teoria/prática	20%	42 medições, seis resumos, gráficos e discussão; máquina da referência pendente
Estratégias exageradas e extrapolação	15%	quatro cálculos explícitos, oito curvas e custos hipotéticos auditáveis
Relatório, recomendação e comunicação	10%	PDF completo, fontes editáveis, recomendações por cliente e roteiro oral

A soma dos pesos é 100%. Não se estima nota: ela depende do professor e das pendências acima, especialmente linguagem e máquina de referência.

Roteiro de apresentação e defesa oral

Apresentação de 8 a 10 minutos. Integrantes: Henry Rodrigues, Rafael Maluf e Matheus.

- Minuto 0–1: explicar o problema, D=65536 e a diferença entre 256 B extras e 2 s no hardware da disciplina. Declarar o caráter provisório de linguagem/par.

- Minutos 1–3: mostrar contrato e invariantes. Heap ordena e compacta; bitmap testa e marca um bit, mantendo primeiras ocorrências. Ambos retornam prefixo de u elementos.

- Minutos 3–4: abrir dados/memoria.txt e evidências GCC. Explicar 48+32+16=96 B, heap dinâmico zero e por que RSS é outro recurso.

- Minutos 4–6: mostrar medianas/dispersões e gráfico de tempo. Identificar duas medições e oito previsões. Explicar o painel de expoentes grandes sem ler seu eixo como segundos.

- Minutos 6–8: escolher um cálculo de cada estratégia exagerada; apontar as correções respeitosas do enunciado.

- Minutos 8–10: recomendar por cliente, executar make test se houver tempo e declarar as pendências externas.

Perguntas prováveis

Por que o bitmap não serve à Embarca se é O(1)? Constante em n pode ser grande: só os bits usam 8192 B, contra 256 B permitidos.

Por que não Quick Sort? A versão recursiva usual usa pilha dependente de n. Não demonstramos um limite de 256 B para ela. Heap iterativo fornece um caminho de chamadas fixo com evidência no build.

Como provam que nenhum único sumiu? O oráculo de testes conta todos os valores de entrada, exige u igual ao número de distintos e confere cada saída uma única vez; também valida a ordem contratada. Ele não é uma terceira implementação entregue ao cliente.

Como compactar sem apagar o futuro? O índice de saída nunca ultrapassa o índice já lido. No heap, o vetor está ordenado antes da compactação; no bitmap, o valor atual é lido antes da escrita.

Os 96 B são universais? Não. São os frames static deste GCC e flags, com margem conservadora. Recompilar para outra ABI exige auditar de novo; interrupções e infraestrutura são reservas separadas.

O bitmap passou em menos de 2 s? Localmente sim, conforme CSV. O requisito da máquina da disciplina permanece pendente.

Por que n=100000 não pode ser todo distinto? Há apenas 65536 valores possíveis. Pelo princípio das gavetas, n>D implica repetição.

Bloom a 1% perde só 1% dos dados necessariamente? Não. A taxa é de falso positivo por consulta inédita sob hipóteses de hash/ocupação; não é uma garantia determinística de erro total nem de saída inteira correta. Um falso positivo pode descartar um valor novo.

Por que o Bloom é maior que o bitmap? Ele é útil para universos enormes/esparsos. Para este domínio pequeno e totalmente conhecido, guardar um bit por valor custa menos que um filtro dimensionado para 65536 distintos a 1%.

A codificação unária é exponencial? Em relação aos bits b de um valor, pode ser $\Theta(2^b)$. Em relação a n com b=16 fixo, o volume é no máximo 65536n bytes neste formato. Serializar não deduplica: ainda é preciso comparar registros.

O power set exige guardar todos os subconjuntos? Não. Enumerar em streaming usa pilha linear; materializar todos os elementos custa $\Theta(n2^n)$. As duas variantes foram distinguidas.

Bogosort sempre leva n! tentativas? Não. Com repetições o esperado é $n!/\prod c_j!$; já ordenado termina na checagem inicial. Com todos iguais, uma checagem basta.

De onde vieram os segundos das oito curvas? São modelos declarados, com 1 ns por operação nominal, multiplicadores e I/O idealizado. Não foram medidos e não oferecem garantia de prazo.

Por que C se a rubrica fala Pascal? A seção 6 permite linguagem à escolha; adotamos C com base nessa seção, documentamos a inconsistência e precisamos confirmar com o professor.